

PRD — Agent + Workflow Management System

Mofa-AI Open-Source Assignment

Project	Agent + Workflow Management System
Author	Sarang · github.com/workwithsarang
Organisation	Mofa-AI
Version	1.0
Date	May 2026
Status	Complete
Repo	github.com/workwithsarang/mofa-ai-workflow

1. Overview

This document covers the requirements, design decisions, and implementation details for the Agent + Workflow Management System I built for the Mofa-AI open-source assignment.

The core idea: users define small text-processing units called **Agents**, then chain exactly two of them into a **Workflow**. Running a workflow pipes user input through Agent A, feeds that output into Agent B, and returns a step-by-step breakdown of what happened.

2. Problem I was solving

Text transformation pipelines — things like normalising input, counting words, reversing strings — are usually hardcoded and scattered across projects. There's no standard way to:

- Define a reusable processing unit
- Compose two units into a pipeline
- See exactly what each step produced
- Do all of this per-user without data leaking between accounts

This system gives that a proper home.

3. Goals

#	Goal
G1	Each user manages their own agents and workflows — complete data isolation
G2	Creating and editing agents requires no code — just a form
G3	Workflows chain Agent A → Agent B where A's output becomes B's input
G4	Running a workflow shows step-by-step output, not just the final result

G5	Every feature is accessible via a REST API documented in Swagger
G6	The whole thing is deployed and reachable without any local setup

4. What I didn't build (out of scope)

- Chains longer than 2 agents (could be a future iteration)
- Real-time streaming of execution
- Sharing agents or workflows between users
- Admin roles or permissions management
- Drag-and-drop workflow canvas
- Usage limits or billing

5. Who uses this

Developer / Evaluator — wants to poke the API, check Swagger, verify that auth and ownership rules work correctly. Needs: predictable HTTP codes, good docs, curl examples.

End user — wants to build agents and wire them into pipelines through the UI without touching any API directly. Needs: clean forms, status indicators, easy run + results view.

Open-source contributor — wants to clone, understand the structure quickly, and maybe add a new agent type. Needs: a clear pattern to follow, `.env.example` so they're not guessing env vars.

6. Feature Requirements

Authentication

ID	Requirement	Priority
F1.1	Register with email + password	Must
F1.2	Passwords hashed with bcryptjs, salt rounds = 10	Must
F1.3	Login returns a signed JWT (7-day expiry)	Must
F1.4	Protected routes require Authorization: Bearer <token>	Must
F1.5	No token → 401 · Bad/expired token → 403	Must
F1.6	JWT stored in localStorage; Axios interceptor auto-attaches it	Must
F1.7	Logout clears token, redirects to login	Must

Agent Management

ID	Requirement	Priority
F2.1	Create agents with: name, type, status, optional inputSchema, optional processingLogic	Must
F2.2	Supported types: UPPERCASE, WORD_COUNT, REVERSE, TRIM	Must

F2.3	Status is ACTIVE or INACTIVE (default ACTIVE)	Must
F2.4	List, view, update, delete — all scoped to the logged-in user	Must
F2.5	Accessing another user's agent returns 404 (not 403 — don't leak existence)	Must
F2.6	Status badge in UI: green for active, grey for inactive	Must

Agent Processing

ID	Requirement	Priority
F3.1	UPPERCASE — converts input to uppercase	Must
F3.2	WORD_COUNT — returns word count as string	Must
F3.3	REVERSE — reverses all characters	Should
F3.4	TRIM — strips leading/trailing whitespace	Should
F3.5	All processing logic in one <code>runAgent()</code> switch-map — adding a new type is one code block	Must

Workflow Management

ID	Requirement	Priority
F4.1	Create workflow with a name and exactly 2 agent IDs	Must
F4.2	Both agents must belong to the creating user	Must
F4.3	List view shows workflow name, Agent A → Agent B labels, and Run button	Must

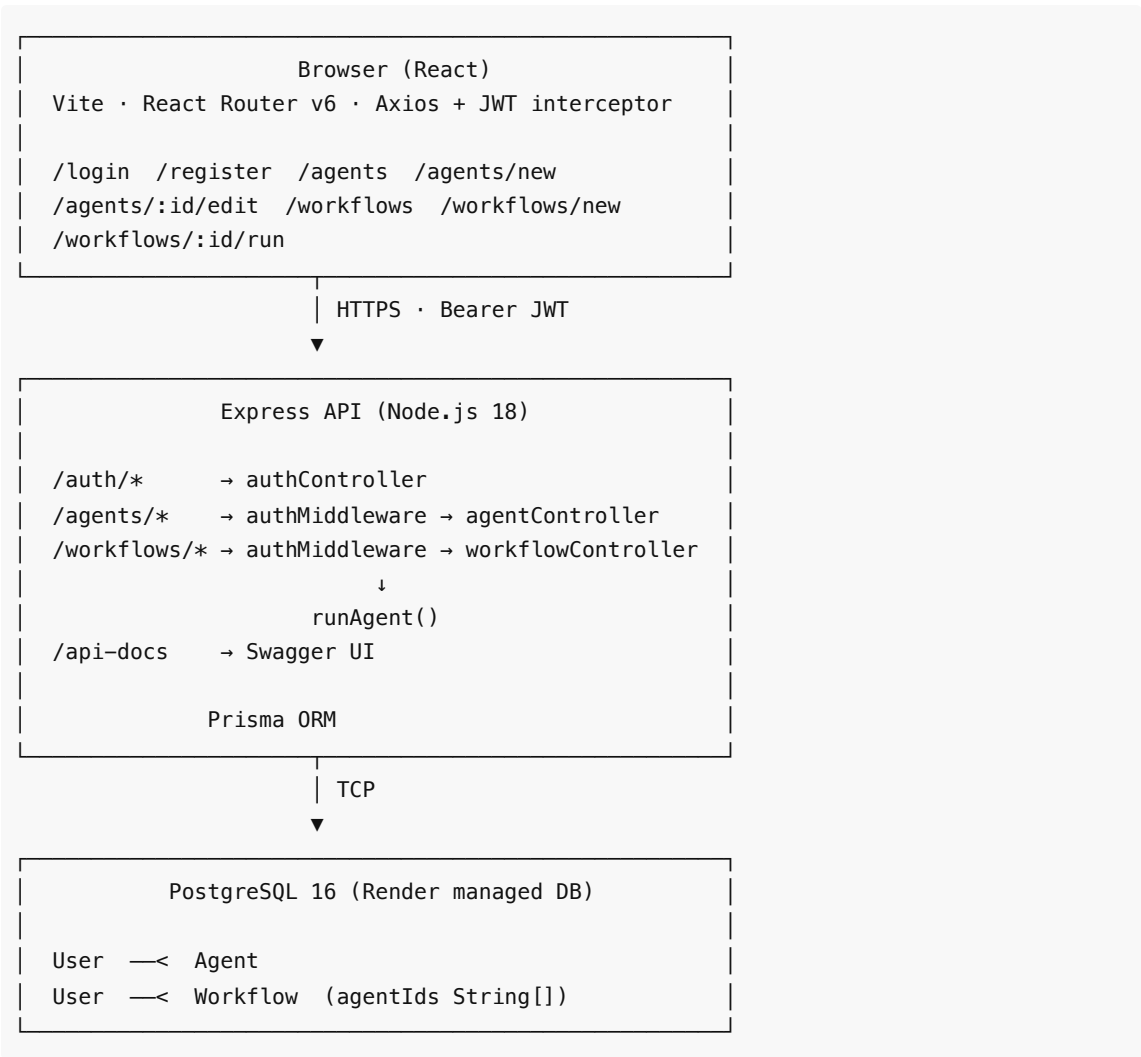
Workflow Execution

ID	Requirement	Priority
F5.1	POST <code>/workflows/:id/run</code> accepts <code>{ input: string }</code>	Must
F5.2	Step 1: run Agent A on input	Must
F5.3	Step 2: run Agent B on Step 1's output	Must
F5.4	Response: <code>{ steps: [...], finalOutput: "..."} </code>	Must
F5.5	UI shows each step in a card + final output highlighted separately	Must

API Docs

ID	Requirement	Priority
F6.1	Swagger UI at <code>/api-docs</code>	Must
F6.2	Every endpoint has request schema, response schema, auth requirement documented	Must

7. Architecture



Database models

User — id, email (unique), password (hashed), createdAt, updatedAt

Agent — id, userId (FK), name, type (enum), inputSchema?, processingLogic?, status (enum), createdAt, updatedAt

Workflow — id, userId (FK), name, agentIds (String[]), createdAt, updatedAt

8. API reference

Method	Endpoint	Auth	Description
POST	/auth/register	No	Register
POST	/auth/login	No	Login → JWT
GET	/agents	JWT	List agents

POST	/agents	JWT	Create agent
GET	/agents/:id	JWT	Get agent
PUT	/agents/:id	JWT	Update agent
DELETE	/agents/:id	JWT	Delete agent
GET	/workflows	JWT	List workflows
POST	/workflows	JWT	Create workflow
GET	/workflows/:id	JWT	Get workflow
POST	/workflows/:id/run	JWT	Run workflow
GET	/api-docs	No	Swagger UI
GET	/health	No	Health check

HTTP codes used:

- 200 success · 201 created · 400 bad input · 401 no token · 403 bad token · 404 not found or not yours · 409 duplicate

9. Security

What	How
Password storage	bcryptjs, salt rounds 10
Password never returned	Prisma queries never select the password field
JWT signing	HS256, JWT_SECRET from env, 7-day expiry
Ownership isolation	Every query filters by userId: req.userId
Existence leaking	Wrong-owner lookups return 404, not 403
Secrets in git	.env is gitignored; .env.example ships instead

10. Frontend pages

Route	Page
/register	Email + password form. Redirects to login on success.
/login	Saves JWT to localStorage on success, sends to /agents.
/agents	Table — name, type, status badge, Edit + Delete per row.
/agents/new	Create form.
/agents/:id/edit	Pre-filled edit form.

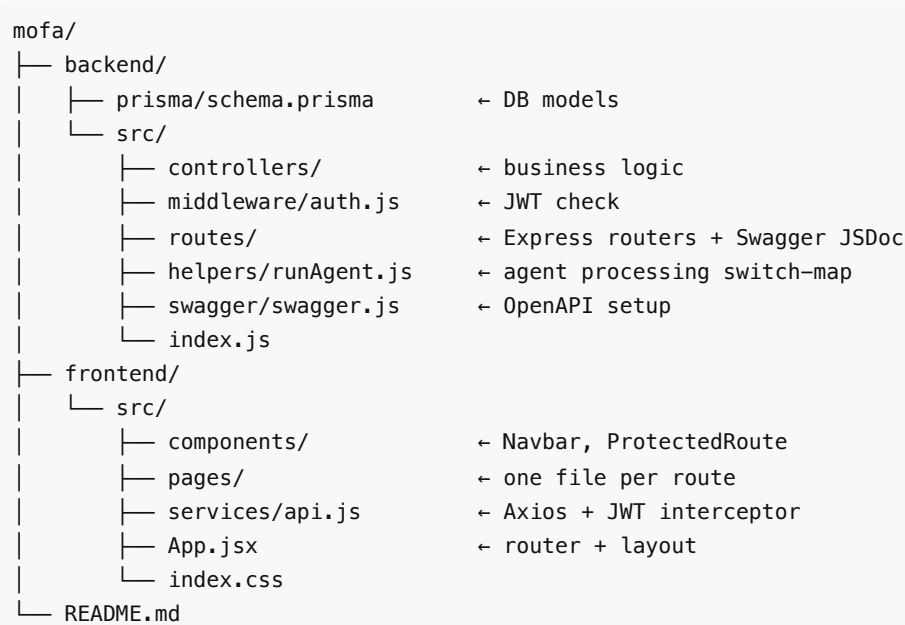
/workflows	Table — name, Agent A → B, Run button per row.
/workflows/new	Name input + 2 agent dropdowns (active agents only).
/workflows/:id/run	Textarea input → step cards → final output box.

All routes except /login and /register redirect to login if no token is present.

11. Deployment



12. Folder structure



13. How to add a new agent type

Say you want to add TITLECASE :

1. Add `TITLECASE` to the `AgentType` enum in `backend/prisma/schema.prisma`
2. Run `npx prisma migrate dev --name add-titlecase`
3. Add one entry in `backend/src/helpers/runAgent.js` :

```
TITLECASE: (input) => input.replace(/\b\w/g, c => c.toUpperCase()),
```

4. Add `'TITLECASE'` to the `TYPES` array in `frontend/src/pages/AgentForm.jsx`

That's it — nothing else needs changing.

14. Known trade-offs

- Workflows are fixed at 2 agents for now. Extending to N agents would need a loop in `runWorkflow()` and a UI that supports variable-length chains — left for a future version.
- `agentIds` is stored as a `String[]` on the Workflow model (PostgreSQL array). This is simple but means there's no FK constraint at the DB level — ownership is enforced in the controller instead.
- Frontend state isn't managed with Redux/Zustand; each page fetches its own data. Fine at this scale, but would need a shared store if the app grew.